Литература: Брайант О'Халлорон Компьютерная архитектура. Взгляд программиста.

Директивы:

- model tiny (для всех нужд достаточно 64КБ)

сейчас плоская модель

- code говорит, что дальше пойдёт код
- 286 - процессор
- org 100h — первые 256 байт - системные нужды дос, поэтому 1 байт кода загружается в 256 ячейку

256₁₀

Затем уже код

Start:

int - системный вызов 21h в ах конкретная функция

печать строки - 9 в старшие разряды

Отладчик: td ctrl+enter

горячие клавиши отладчика: F7 - trace

alt+F5 - посмотреть экран

Задание: до пятницы написать ещё один hello world, расположив её правильно и неправильно

с помощью hex editor поменять 3Ч 09 → 3Ч02

ASM define

FnPutS equ 09h — универсальнее

FnPutChar = 02h

тогда можно: mov ah, FnPutS ; DOS print str function

Функции можно добавить в DOS Func.asm

в asm файле с прогой:

include DOS Func.asm 7-мя проходный ассемблер

.Exit

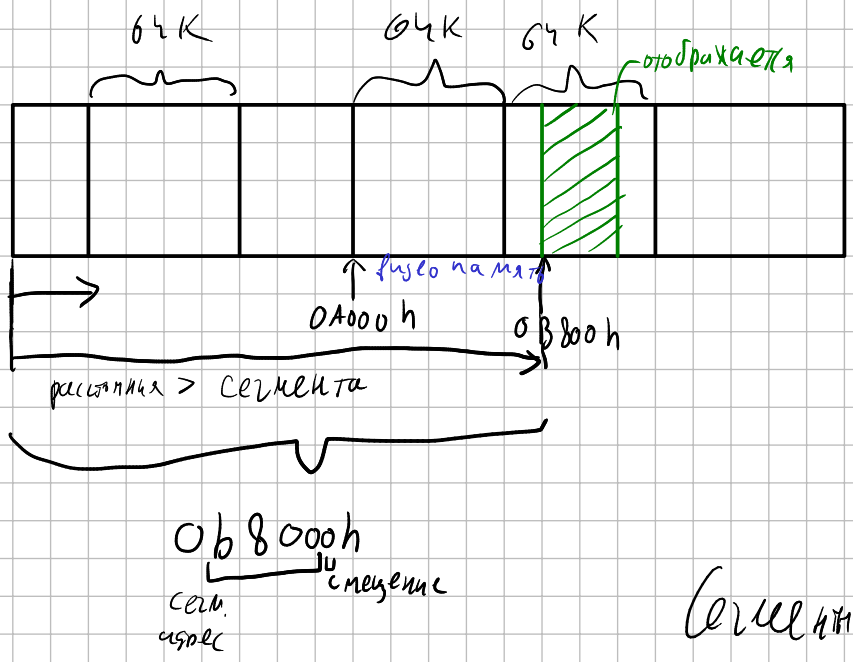
```
macro
nop
mov ax, 4c00h
int 21h
endm
nop
```

начало и конец макроса, помогает их определять в дебаггере

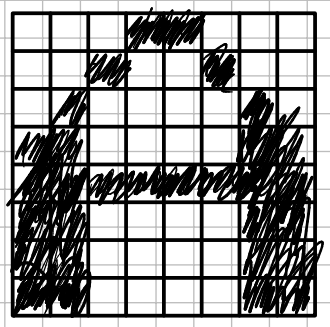
или

```
mov ax, ax
:
mov bx, bx
```

vmem.asm



в текстовом режиме в видеопамяти есть область памяти - знакогенератор



Сегментный адрес - старший байт адреса

лин адрес = сегм. адрес • 16 + смещение

$\underbrace{Seg}_{16\ бит} \cdot 16 + \underbrace{Ofs}_{16\ бит}$ 20 бит в адресе

в регистрах, по которым можно индексироваться

сегментные регистры

в коде 2 байта 3 байта

DS	data segment = адрес, где запустилась программа [bx] = ds:[bx]
SS	stack segment если поменять, то push, pop будут писать куда-то
CS	code segment вообще лучше не трогать
ES	extended segment его можно юзать!

стек растёт в обратном направлении, чтобы размер был только нужного размера

```
Start:  mov bx, 0b800h
        mov es, bx        нельзя константу напрямую

        mov bx, (5·80 + 80/2)·2
        mov byte ptr es:[bx], 65d ; 'A'
                ; не понимает, 2 байта или 1 байт
```

раскрасим букву, изменив след. байт

; байт атрибута: Bbbb Fffff

7 6 5 4 3 2 1 0

Bbbb: мигание буквы, фон (rgb)

Fffff: i rgb — цвет буквы; i — буква

повышенная интенсивность

её мно-красный: 0100

жёлтый: 1100

```
        move byte ptr es:[bx+1] чтобы - ярко-жёлтый на красном
```

10-е прерывание — смена видеорежима

jo — jump if overflow

<u>Задание:</u> нарисовать рамочку типа "сохранить как"! Рисовать лучше построчно

1 символ — много символов — 1 символ

Функции.

```
Start:  jmp Main
```

ГЛАВНОЕ - ДОКУМЕНТАЦИИ!

```
; Writes a symbol to video memory
; Entry:  AH - color attribut
;         AL - Symbol
;         BX - offset to video mem
; Assumes: ES = 0b800h
; Destroy: BX
```

WriteSym

proc — директива образует метку, лучше чем : для ф-ций

mov es:[bx], al
inc bx
mov es:[bx], ah } и обычный эффект

ret

можно имя написать

endp — директива, конец процедуры

Локальные метки

делаются отн. процедуры, локальная, если начинается с ??

или можно поменять префикс лок. меток директивой: locals @@

например

директива jumps позволяет условно прыгать куда угодно,

использ jmp

CLD DF=0 — DI↑, DF=1 — DI↓, чтобы DF=1 STD
set direction flag

STOSB — ES:[DI++] = AL
store string of bytes

STOSW — ES:[DI] = AX; DI += 2

STOSD — 32 бита

STOSQ — 64 бита

MOV CX, что-то

NEXT: STOSW
LOOP NEXT (пока CX ≠ 0)

или

MOV CX, что-то

REP STOSW ; while (CX--) { ES:[DI] = AX; DI += 2 }

LODSB — AL = ds:[SI++] DF тоже важен
(по умолчанию)
load string of bytes

MOVSB ES:[DI++] = ds:[SI++]
move string of bytes

REP MOVSB — memcpy memmove что делать при overlap памяти? надо контролировать справа-налево

CMPSB — cmp ds:[SI++], ES:[DI++]

REP CMPSB — бессмысленно

REPE CMPSB — while (cx-- && ES:[DI++] == ds:[SI++])

REPNE memcmp ≠

SCASB — CMP AL, ES:[DI++]

strlen, memchr

XOR AL, AL

REPNE SCASB — вычислить длину строки по CX

сначала проверка, потом CX--

если начинать с нуля, то NEG

но мы с -1, поэтому NEG -1

НАПИСАТЬ strlen, MEMCHR, MEMSET, MEMCPY, MOVMEM, MEMCMP со стек фрейтом, что-то cdecl, что-то pascal

Передача параметров

Func (0x10, 0x20, 0x30)

Func (P(0x10), P(0x20), P(0x30))

порядок вычисления неопр.

Паскалевский вызов

push 10h
push 20h
push 30h

pascal перед названием ф-ции

Сишный вызов — cdecl

push 30h
push 20h
push 10h

влияет на ф-ции с переменным кол-вом параметров

в printf 1-й аргумент — форматная строка

определяет все остальные аргументы

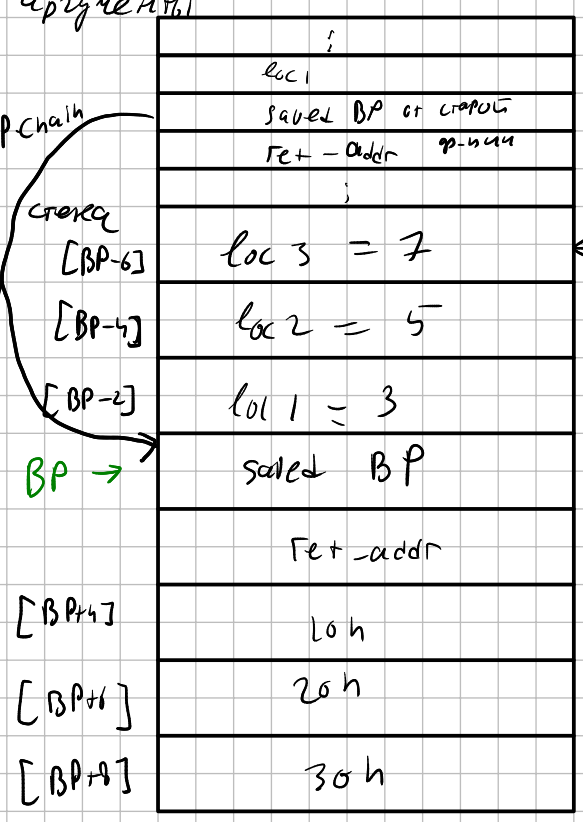
cdecl

push 30h
push 20h
push 10h
call func
add SP, 6, если 2 пара, то быстрее pop

функция не очищает сама стек от лишней, если var params

pascal

push 10h за обсчитанные
push 20h ответчик
push 30h сама ф-ция
call func



func proc

mov ax, [SP+2]

push cx надо запоминать все push'ы

mov cx, [SP+2] ; ??? (ret addr)

func proc

push BP

mov BP, SP ; а что если эта функ вызвана из другой?

mov ax, [BP + 4]

mov cx, [BP + 4] ; !!!

...

pop BP

RET в PASCAL, проц. сразу корректирует стек

локальные переменные тоже в стек но другую сторону от BP

в 64 бит - Stdcall, 1-ые 6 параметров в регистрах, остальные в стеке

ControlString db '[ ]||-||[ ]'

frame.asm 40 15 70 { 0 1 2 3 hello world

wxh color

* 1 2 3 4 5 6 7 8 9

Взаимодействие с клавиатурой.

Замыкание → Контроллер → Контроллер на матрице (PPI-Peripheral) → Регистр данных

в матричной схеме мало клавиш можно нажать
в прямой схеме можно сколько угодно

Контроллер выдает скан-код — порядковый номер на стандартной клавиатуре 84 клавиши

Esc = 1
1 = 2

A = a
2 = @ — одна и та же клавиша
⋮

Скан кодов всего 127, т.к. скан код нажатия ≠ отпускания, записывается в старшем бите

в современных клавах для новых клавиш посылают посл-ть скан-кодов, начиная с E0 или E1 — никакая буква.

Процессор чекает регистры.

Порты ввода/вывода. — 60h — регистр данных клавиатуры
например можно установить скорость автоповтора.
запрограммировал в самом низкоуровневом контроллере

Там лежит самый последний скан-код

как читать?
как следить?

in

polling — опрос портов (хреновый метод)

Контроллер на матери может послать сигнал о том, что клавиатура дала сигнал, чтобы прервать проц.
этот сигнал передаётся контроллеру прерываний. Он разруливает вопросы приоритета прерываний. Interrupt request level IRQ
Таймер: IRQ 0, Клава: IRQ 1
высший приоритет

проц получает сигнал о прерывании и номер IRQ

в оперативке с 0h адреса лежит таблица векторов прерываний (адреса ф-ций)

проц отвлекается от текущей задачи: вызывает ф-цию
interrupt flag = 0 блокирует прерывания, кроме прерывания отключения питания

8 + IRQ level — ячейка в таблице прерываний
посмотреть в help, что в 0–7

0 прерывание при делении на 0 или если рез. деления не помещается

Как функцию вызывать?

прерывание может быть в другом сегменте, поэтому, чтобы вернуться надо ещё докинуть сегмент, также сохраняются флаги сначала флаги, потом сегмент, потом смещение

Кто обработчик написал? — Фрейтверкс

Где код? — Биос

Как до биоса достучаться? F000h — FFFFh

Мы обработчик!

1. Считать скан-код
2. Преобразовать в ASCII-код. Надо проверить предыдущий скан-код, shift ли это? Но вызовы независимы.

по адресу 417h флаги клавиатуры.

3. Надо сохранить в буфер. Младший байт — буква, старший — скан-код

адрес — 41eh

4. Клавиатуре надо сказать, что мы прочли скан-код.

61 порт — порт состояния PPI, старший бит — контроль клавиатуры
1 — запрет
0 — разрешить

20h — порт контроллера прерываний, надо в 20h порт 20h загнать

5. Пора возвращаться iret — interrupt return

зачем детектор вызывает int 16h, 0 — функция (считывание) возвращает последний ASCII код или ждёт, если нет

1-я функция никого не ждёт, сигнализирует, что очередь пустая

Эксперименты.

почему ret выходит из прога?

```
.model tiny
.code
.286
org 100h

Start:
    push 0b800h
    pop  es
    mov  bx, (80 * 5 + 40) * 2
    mov  ah, 4eh

Next:
    in   al, 60h ; read data from keyboard controller
    mov  es:[bx], ax

    cmp  al, 11d ; scan-code of 0
    jne  Next

    mov  ax, 4c00h
    int  21h

end Start
```

Подмена обработчика прерываний.

надо будет завершить прогу без очистки памяти
int 31h terminate stay resident

Закинуть обработчик — 2 команды, между которыми может произойти прерывание
cli — запретить int

mov ax, 3508h
int 21h
mov oldoffset, bx
push

```
.model tiny
.code
.286
org 100h

Start:
    mov  ax, 3509h
    int  21h
    mov  Old09Offset, bx
    mov  bx, es
    mov  Old09Seg, bx

    mov  ax, 0
    mov  es, ax
    mov  bx, 4 * 09h ; 4 байта - вектор прерывания, 09h - номер прерывания
    cli ; запретить прерывания
    mov  es:[bx], offset My09
    mov  ax, cs
    mov  es:[bx + 2], ax
    sti ; разрешить прерывания

    mov  ax, 3100h ; завершение без освобождения
    mov  dx, offset GrandFinale
    shr  dx, 4

    inc  dx ; на всякий, вдруг дос съест последний байт

    int  21h

My09 proc
    push ax bx es ; macro in tasm

    mov  ax, 0b800h
    mov  es, ax

    mov  bx, (80 * 5 + 40) * 2
    mov  ah, 4eh ; yellow on red

    in   al, 60h ; read data from keyboard controller
    mov  es:[bx], ax

    pop  es bx ax

            db 0eah ; far jump. After our handler we call the original.
Old09Offset dw 0
Old09Seg    dw 0
endp

GrandFinale: ; end of program to know size

end Start
```

Чтобы печатать надо переменную создать счётчик, но ds какой?
поэтому через cs

Задание: по горячей клавише показывать в верхнем левом углу выдавать значения регистров. Можно по таймеру рисовать регистры